◇ IPC Mechanisms (POSIX vs. System V)

We'll explore all IPC mechanisms with **detailed theory, advanced examples, real-world use cases, and performance comparisons**.

---

1 Pipes (Anonymous & Named)

◇ Overview

A pipe is an **unidirectional communication channel** used between related processes.
- **Anonymous Pipes**: Used only between parent and child processes.
- **Named Pipes (FIFO)**: Used between **any two processes**, even unrelated ones.

◇ System V Pipes (Anonymous)

- Uses `pipe()` to create a pipe.
- Only works between **parent and child** processes.

◇ Example: Anonymous Pipe (System V)

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int fd[2];  // File descriptors for the pipe
    pipe(fd);   // Create pipe

    pid_t pid = fork();
    if (pid == 0) {  // Child process
        close(fd[1]); // Close write end
        char buffer[20];
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    } else {  // Parent process
        close(fd[0]); // Close read end
        write(fd[1], "Hello Child!", 12);
        close(fd[1]);
    }
    return 0;
}
```

◆ **Real-World Use Case**: Shell piping (`ls | grep .c`) and process data transfer.

◇ POSIX Named Pipes (FIFO)

- Created using `mkfifo()`.
- Allows **unrelated processes** to communicate.

◇ Example: POSIX Named Pipe

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_PATH "/tmp/my_fifo"

int main() {
    mkfifo(FIFO_PATH, 0666);  // Create named pipe

    pid_t pid = fork();
    if (pid == 0) {  // Child Process
        int fd = open(FIFO_PATH, O_RDONLY);
        char buffer[100];
        read(fd, buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd);
    } else {  // Parent Process
        int fd = open(FIFO_PATH, O_WRONLY);
        write(fd, "Hello FIFO!", 11);
        close(fd);
    }
    return 0;
}
```

◆ **Real-World Use Case**: Logging servers, inter-process communication in daemons.

◇ Performance Comparison

Feature	System V Pipes	POSIX Named Pipes
Scope	Parent-child	Any process
Persistence	Disappears after process exit	Exists until deleted
Efficiency	Faster (in-memory)	Slightly slower (file-based)

2 Message Queues

◇ Overview

- Message queues allow structured **message passing** between processes.
- **System V** queues use integer keys.
- **POSIX** queues use **named handles** (`mq_open()`).

◇ System V Message Queues

- Uses `msgget()`, `msgsnd()`, and `msgrcv()`.
- Requires **manual cleanup** (`msgctl()`).

◇ Example: System V Message Queue

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg {
    long type;
    char text[100];
};

int main() {
    key_t key = ftok("queuefile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg message;

    message.type = 1;
    strcpy(message.text, "Hello System V IPC!");

    msgsnd(msgid, &message, sizeof(message), 0);
    printf("Message Sent\n");

    return 0;
}
```

◆ **Real-World Use Case**: Logging, real-time data processing.

◇ POSIX Message Queues

- Uses `mq_open()`, `mq_send()`, `mq_receive()`.
- Automatically managed with **file descriptors**.

◇ Example: POSIX Message Queue

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

#define MQ_NAME "/posix_mq"

int main() {
    mqd_t mq = mq_open(MQ_NAME, O_CREAT | O_WRONLY, 0644, NULL);
    mq_send(mq, "Hello POSIX IPC!", 16, 0);
    mq_close(mq);
    return 0;
}
```

◆ **Performance Comparison** | Feature | System V | POSIX | |———|———|——-| | **Naming** | Integer keys | Path-based | | **Persistence** | Requires manual deletion | Auto-cleanup | | **Efficiency** | More overhead | Faster |

---

3 Shared Memory

◇ Overview

- Most **efficient IPC** (direct memory access).
- **System V** uses `shmget()` and **POSIX** uses `shm_open()`.

◇ System V Shared Memory

Example: System V Shared Memory

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *)shmat(shmid, NULL, 0);

    strcpy(str, "Shared Memory Example");
    printf("Data written: %s\n", str);

    shmdt(str);
```

```
    return 0;
}
```

◇ POSIX Shared Memory

Example: POSIX Shared Memory

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

#define SHM_NAME "/posix_shm"

int main() {
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, 1024);
    char *ptr = mmap(NULL, 1024, PROT_WRITE, MAP_SHARED, fd, 0);

    strcpy(ptr, "POSIX Shared Memory Example");
    printf("Data written: %s\n", ptr);

    munmap(ptr, 1024);
    close(fd);
    return 0;
}
```

🔹 **Performance Comparison** | Feature | System V | POSIX | |———|———|——-| | **Management** | Manual cleanup | Auto-managed | | **Access Speed** | Fast | Slightly faster | | **Ease of Use** | Complex | Simpler API |

Semaphores

◇ Overview

- Used for **process synchronization**.
- **System V** uses `semget()`.
- **POSIX** uses `sem_open()`.

◇ System V Semaphore

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int main() {
    key_t key = ftok("semfile", 65);
    int semid = semget(key, 1, 0666 | IPC_CREAT);
    printf("Semaphore ID: %d\n", semid);
    return 0;
}
```

◇ POSIX Semaphore

```c
#include <stdio.h>
#include <fcntl.h>
#include <semaphore.h>

#define SEM_NAME "/posix_sem"

int main() {
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    printf("POSIX Semaphore created\n");
    sem_close(sem);
    return 0;
}
```

◆ **Performance Comparison** | Feature | System V | POSIX | |———|———|——-| | **Ease of Use** | Harder | Easier | | **Performance** | Similar | Similar | | **Portability** | Limited | Cross-platform |

Conclusion

- **Use POSIX IPC** for **new applications** (simpler, better managed).
- **Use System V IPC** for **legacy systems** requiring backward compatibility.